

2026年度 筑波大学

数理工学モデル化演習(岩永担当1回目)

Python言語と数理最適化による問題解決:入門

担当:岩永二郎

株式会社エルデシュ 代表取締役

電気通信大学 特任教授

iwanaga@erdos-the-book.com

はじめに

自己紹介

■ 岩永二郎, Ph.D.(社会工学)

筑波大学 高野祐一先生の研究室

- ✓ 株式会社エルデシュ 代表取締役
- ✓ 電気通信大学 特任教授
- ✓ 筑波大学 非常勤講師 / 上智大学 非常勤講師



ベンダー(8年)

2008年～

(株)NTTデータ数理システム

- ・数理科学全般
- ・プログラミング
- ・受託開発 / 分析コンサル

事業会社(3年)

2016年～

Retty(株)

- ・数理科学のサービス接続
- ・データビジネス事業開発
- ・大学非常勤講師

DSコンサル(6年)

2019年～

(株)エルデシュ

- ・AIシステム開発
- ・データ分析/データ活用支援
- ・技術顧問/経営支援

▶ 2024年から電通大に着任

はじめに

- ゴール
 - ✓ Python言語を用いて数理最適化問題が解ける
 - ✓ 実務で利用する数理最適化の考え方を身につける
- 出欠は respon で15:00～18:00の間に行う。
- manaba を利用する
 - ✓ 連絡は「コースニュース」で行う
 - ✓ 講義情報は「コースコンテンツ」にアップロードする
 - ✓ 課題は「レポート」から提出する
- その他の質問は iwanaga@erdos-the-book.com へ



岩永二郎著、石原響太著、西村直樹著、田中一樹著
Pythonではじめる数理最適化
-ケーススタディでモデリングのスキルを身につけよう-

執筆の背景

数理最適化業界の課題

- ① 理論と実装の教育を受ける環境が少ない
- ② 技術を実務へ適用する機会が少ない
- ③ 性能の良いライブラリが手に入らない

時代の動き

- ✓ データサイエンティストの増加
- ✓ 性能の良い無償のオープンソースライブラリの出現
- ✓ 数理最適化エンジニアに要求される仕事の変化

講義のスケジュールと内容

- 岩永担当1回目 4.21(火)15:15-18:00
 - ✓ 教科書「Pythonではじめる数理最適化」1章・2章相当の内容
 - 数理モデル
 - 線形計画問題
 - 整数計画問題
- 岩永担当2回目 4.28(火)15:15-18:00
 - ✓ 教科書「Pythonではじめる数理最適化」3章
- 岩永担当3回目 5.12 (火)15:15-18:00
 - ✓ 教科書「Pythonではじめる数理最適化」4章～7章から1つ選択

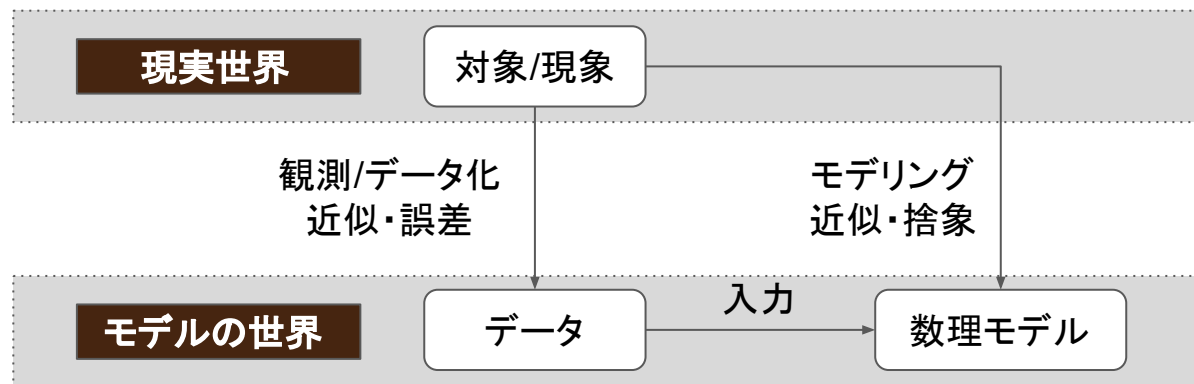
本日の講義

- 数理モデル入門
- 数理最適化チュートリアル
- Python数理最適化入門(ハンズオン)
- 演習

数理モデル入門

「現実世界」と「モデルの世界」

学問上「データ」と「数理モデル」を分けるが、
モデルの世界では「データ」も「数理モデル」も現実世界の“近似”



「データ」を現実世界の“近似”と考える背景

■ 次の「対象/現象」を定量化した「データ」との関係を考える

- ✓ 「学力」を定量化したデータが「試験の点数」
- ✓ 「暑さ、寒さ」を定量化したデータが「気温」
- ✓ 「会社の業績」を定量化したデータが「KPI」

■ 問い

- ✓ 「試験の点数」が上がれば「学力」が向上したと言えるか？
- ✓ 「気温」が上がっても「湿度」次第で「暑さ」は変わる？
- ✓ 「KPI」ハックして数字を伸ばしても会社はよくなるか？

現実世界(対象/現象)とデータにはギャップがある

モデルとは？

■ モデルには**模型**と**見本**の意味がある

✓ 【模型】本物を近似したモデルを通して理解しよう

- 例) プラモデル
- 例) フィギュアモデル



✓ 【見本】見本となるモデルに従って意思決定/判断しよう

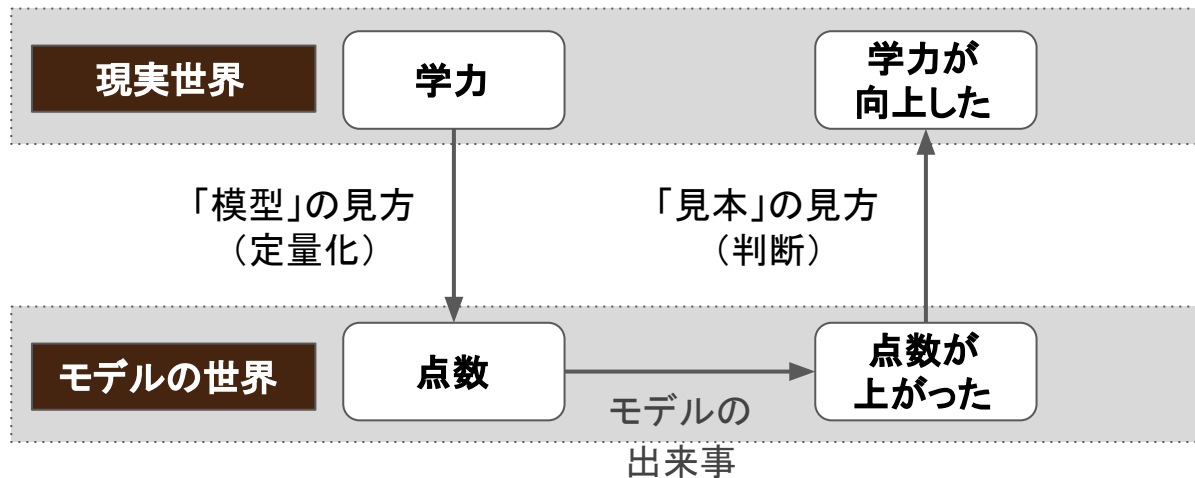
- ファッションモデル
- ロールモデル
- モデル校



モデルは、**理解のための近似** であり、**従うための基準**

現実世界とモデルの世界の対応

■ 学力と点数の例



実務では「模型」と「見本」の見方をする際に
2回の近似を行う

「模型」と「見本」の近似を意識できない悲劇

■ 数理モデルの罠

「今野浩著、金融工学20年～20世紀エンジニアの冒険」

経済学者は、様々な要因が複雑に絡み合っている経済現象に大胆な単純化を施し、その**本質部分を暴き出して理論をつくる**。理論をつくった人たちは、現在の経済現象を十分な精度で説明できるとは限らないことを知っており、理論の現実への応用については十分な謙虚さを備えている。しかし、**教科書を通じてこの理論を学んだ人々の中には過信する人が現れ、現実はこの理論通りに動いているに違いない**と信じる。

現実世界とモデルの世界のギャップが
数理モデルの現実世界への適用可否を決める

数理最適化モデルとは？

■ 数理最適化モデルの特徴

- ① 複数の選択肢から最適な選択肢を見つける数理モデル
- ② 組合せ的な難しさやトレードオフの難しさがある

■ 数理最適化問題の例

【最短経路問題】

A地点からB地点に向かう際の
最短経路を求める

<https://sites.google.com/erdos-the-book.com/opt-app-traveling-salesman>



【ナップサック問題】

予算内で最も得する荷物の
組合せを求める

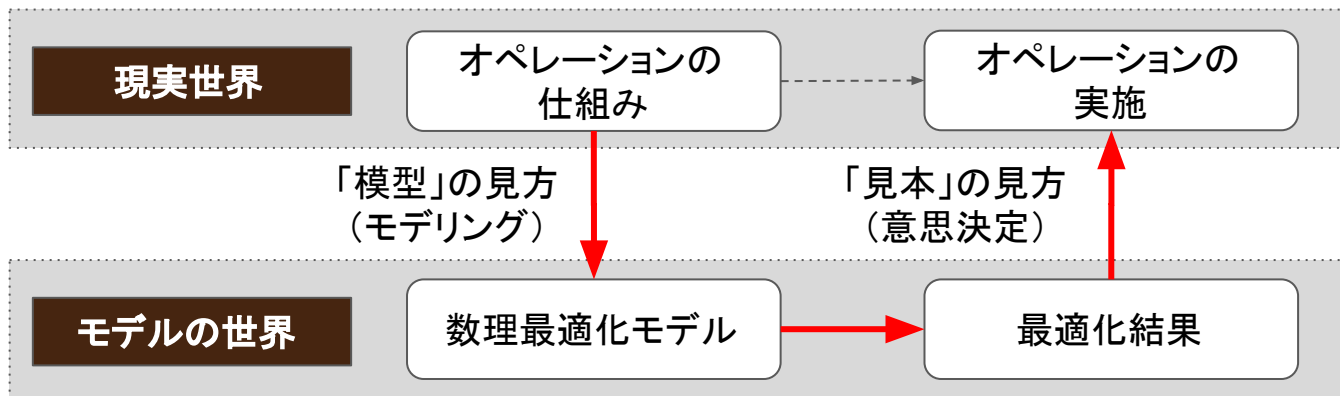
<https://sites.google.com/erdos-the-book.com/opt-app-napzak>



現実世界への数理最適化モデルの適用

■ 数理モデルの模型と見本

- ✓ 【模型】現実を近似、簡略化した模型が数理モデル
- ✓ 【見本】数理モデルを見本に現実で意思決定/判断をする



「模型」と「見本」の見方をする際のギャップに注意して
オペレーションを改善する

数理最適化チュートリアル

数理最適化とは

- **数理最適化**とは、ある条件に関して最も良い元を、利用可能な集合から選択することをいう。[出典:wikipedia]
- 数理最適化問題の表現方法

集合表現

$$\min. f(x) \quad s. t. \quad x \in X$$

条件表現

$$\min. f(x) \quad s. t. \quad \phi(x)$$

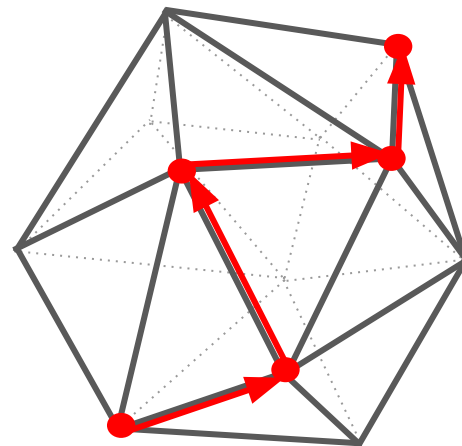
行列表現

$$\min. f(x) \quad s. t. \quad Ax \leq b$$

線形計画法の幾何学的イメージ

- **幾何学的解釈:**
凸多面体の内部/周が解
- **定理:**
最適解は多面体のいずれかの頂点
- **アルゴリズム: 単体法 (Danzig, 1947)**
適当な頂点から探索をはじめ
目的関数が改善される頂点へ移動を
繰り返すと最適解にたどり着く

単体法のイメージ



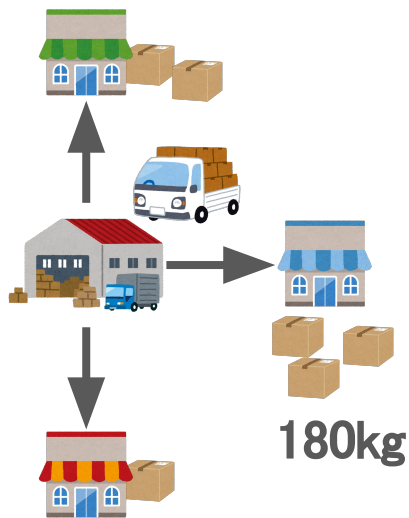
数理最適化でできること

- 数理最適化を活用すると決められたルール(制約)の中で基準(目的関数)に従って賢く選べるようになる
 - ✓ 【補足1】ルールや基準が曖昧だと相性が悪いことがある
→ ルールや基準が明確になれば数理最適化がハマる
 - ✓ 【補足2】最適な選択(1番)でなくても十分なメリットがある
→ 最適解にこだわらず、オペレーションが改善するかどうかが大事

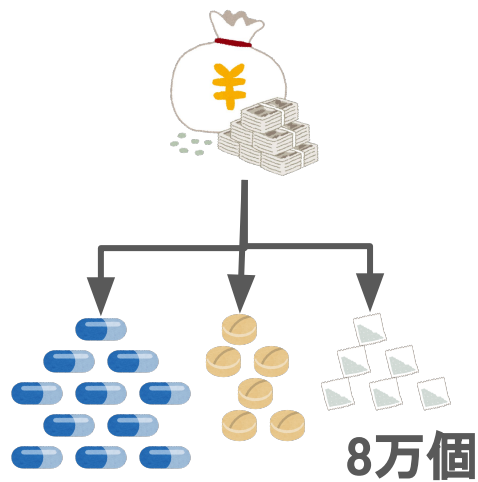
数理最適化による意思決定：数量の決定

数量の決定とは、**量・個数・割合** などを決めること

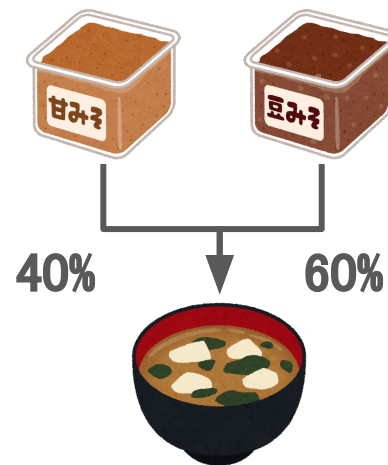
配送量の決定



生産個数の決定



配合率の決定

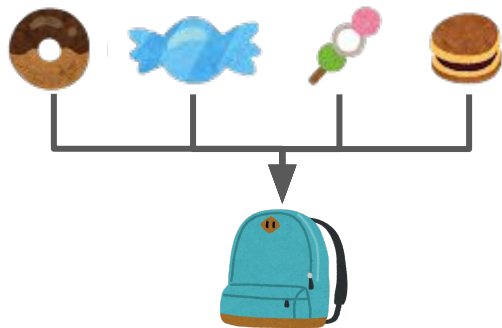


数理最適化による意思決定：二値の決定

二値の決定とは、やる(1)・やらない(0) を決めること

持ち物の決定

どれ をリュックに
入れる(1)か入れないか(0)



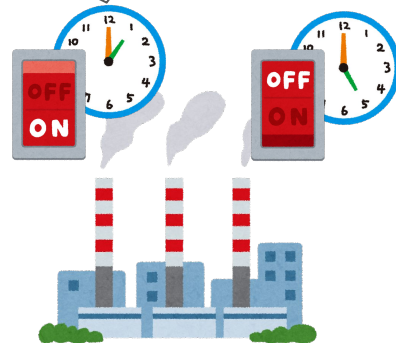
シフトの決定

誰 が いつ シフトに
入る(1)か入らないか(0)



発電の決定

何時 にスイッチを
入れる(1)か切るか(0)



なぜ、いま数理最適化なのか？

技術の進歩

計算機の性能向上とアルゴリズムの成熟により
大規模かつ実務的な問題が解ける時代に

最適化-Ready 時代への突入

データ整備とデータ連携に加え、AIによる予測も
揃い、最適化に必要な情報基盤が整った

意思決定の 大規模化と複雑化

人間の勘や経験だけでは対応しきれない
複雑で大規模な意思決定が増加

持続可能な 社会の実現

SDGsや脱炭素社会の実現に向け、環境負荷の低減や
資源配分の最適化が求められている

モデリング言語 PuLP とソルバー CBC

- **PuLP** はCOIN-OR(※1)で開発しているモデリング言語
→ 問題を**定式化(表現)**する部分のライブラリ

- **CBC** はCOIN-ORで開発しているソルバー
→ 問題を**解く(解を探索)**部分のライブラリ

- インストール

```
$pip install pulp
```

- PuLPをインストールすると
自動でCBCもインストール
→ ユーザーはCBCを意識せず利用可能

【テクニカルノート】

最近では、CBCではなく、SCIPやHiGHSなどのソルバーも利用されている。

※1 COIN-OR: Open Source for the Operations Research Community

数理最適化モデルの仕組み

- 数理最適化モデルは制約プログラミングで記述する

制約プログラミングはプログラミングパラダイムの一つである。
制約プログラミングにおいては、**変数間の関係を制約という形で記述**することによりプログラムを記述する。【出典:Wikipedia】

→ 広く知られているプログラミングパラダイムと異なることに注意

- 数理最適化モデルの4つの部品: 定数・変数・目的関数・制約式

関数		数理最適化モデル
入力	定数	制約式と目的関数の係数・定数項
出力	変数	決定したい値
関数	目的関数	最適化したい指標の記述
定義	制約式	変数間の関係や範囲の記述

【参考】MPSファイル

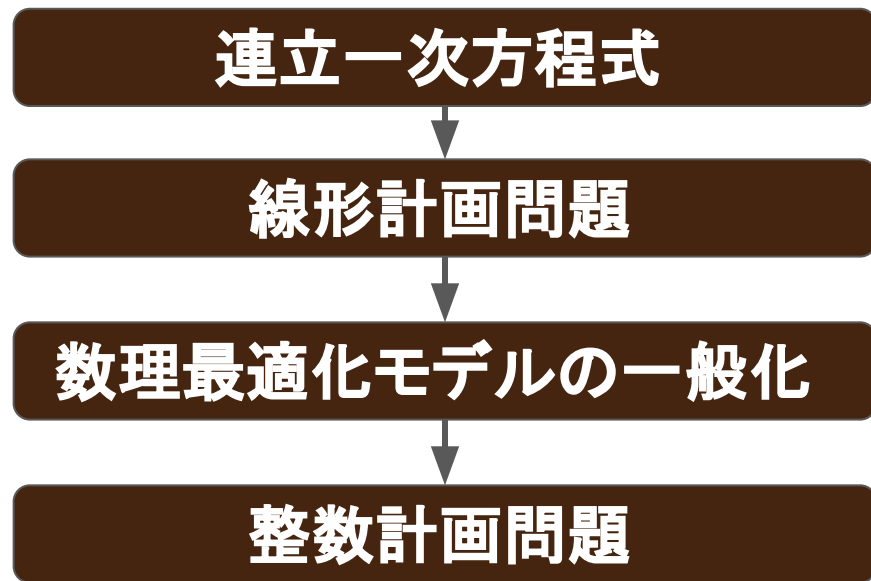
```
*SENSE:Maximize
NAME          MODEL
ROWS
  N   OBJ
  L   C0000000
  L   C0000001
COLUMNS
  X0000000  C0000000  1.0000000000000e+00
  X0000000  C0000001  2.0000000000000e+00
  X0000000  OBJ       1.0000000000000e+00
  X0000001  C0000000  3.0000000000000e+00
  X0000001  C0000001  1.0000000000000e+00
  X0000001  OBJ       2.0000000000000e+00
RHS
  RHS      C0000000  3.0000000000000e+01
  RHS      C0000001  4.0000000000000e+01
BOUNDS
ENDATA
```

PuLPでは次の記述により最適化計算時に作成される MPSファイルを削除しないようにできる

```
solver = pulp.PULP_CBC_CMD(keepFiles=1)
status = prob.solve(solver)
```

Python数理最適化入門

Python数理最適化入門



連立一次方程式を解く

連立一次方程式

【問題】1個**120円** のリンゴと1個**150円** のナシを合計**10個** 買った。代金が**1,440円** のときリンゴとナシはそれぞれ何個買いましたか。

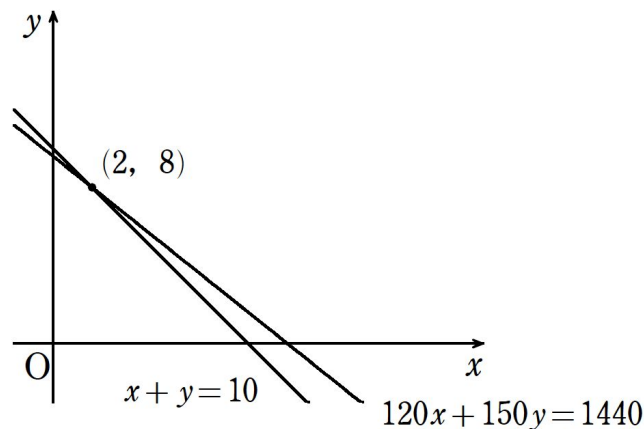
【解答】リンゴの個数を x 、ナシの個数を y とすると

$$x + y = 10$$

$$120x + 150y = 1440$$

これを解いて、 $x = 2$ $y = 8$

よって、リンゴを**2個**、ナシを**8個** 買った。



PuLPによる実装

```
import pulp

# 連立一次方程式は目的関数はないが、最大化問題として数理モデルを仮定義
prob = pulp.LpProblem('SLE', pulp.LpMaximize)

# 変数の定義
x = pulp.LpVariable('x', cat='Continuous')
y = pulp.LpVariable('y', cat='Continuous')

# 制約式の定義
prob += 120 * x + 150 * y == 1440
prob += x + y == 10

# 最適化計算の実行
status = prob.solve()

# 最適化結果の確認
print('Status:', pulp.LpStatus[status])
print('x=', x.value(), 'y=', y.value())
```

```
status: Optimal
x=2.0 y=8.0
```

線形計画問題を解く

生産計画問題(領域の最大・最小)

【問題】原料 m と n を用いて、製品 p と q を製造する。
次の表を参考に利得を最大にする製品 p と q の生産量をもとめよ。

必要量	原料 m	原料 n
製品 p	1	2
製品 q	3	1

原料	在庫
m	30
n	40

製品	利得
p	1
q	2

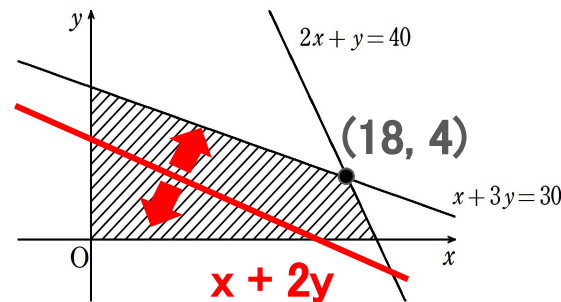
【解答】製品 p の生産量を x [kg]、製品 q の生産量を y [kg]とすると

$$x + 3y \leq 30$$

$$2x + y \leq 40$$

$$x \geq 0, y \geq 0$$

のもとで $x + 2y$ を最大化し $x = 18$ 、 $y = 4$ から
製品 p を18[kg]、製品 q を4[kg]生産すればよい。



PuLPによる実装

```
import pulp

# 最大化問題として数理モデルを定義
prob = pulp.LpProblem('LP', pulp.LpMaximize)

x = pulp.LpVariable('x', lowBound=0, cat='Continuous')
y = pulp.LpVariable('y', lowBound=0, cat='Continuous')

prob += 1 * x + 3 * y <= 30
prob += 2 * x + 1 * y <= 40

# 目的関数の定義
prob += x + 2 * y

status = prob.solve()

print('Status:', pulp.LpStatus[status])
print('x=', x.value(), 'y=', y.value())
print('obj=', prob.objective.value())
```

```
status: Optimal
x=18.0 y=4.0
obj=26.0
```

数理最適化モデルの一般化

これまでの実装の問題点

■ 問題点1: 変数・制約・目的関数をベタ書きしている

数万変数の問題では、変数の定義を数万行書いたり、
数万変数を含む長い制約式や目的関数を書かなければならない。

■ 問題点2: 入力データの変更に弱い

入力データが変更になると、変数を再定義したり、
制約式や目的関数に含まれる係数を全て書き換えなければならない。

→ 数理最適化モデルの一般化：

変数や制約式をまとめて定義することで **コード量を減らせる**。

→ データとモデルの分離 により、問題が変更しても、

入力データの差替えで済むため **汎用性を高める** ことができる。

数理最適化モデルの一般化①

- データのキーとなる要素を**集合**として定義する

【問題】原料 m と n を用いて、製品 p と q を製造する。

次の表を参考に利得を最大にする製品 p と q の生産量をもとめよ。

必要量	原料 m	原料 n
製品 p	1	2
製品 q	3	1

原料	在庫
m	30
n	40

製品	利得
p	1
q	2

- 集合

✓ **製品 p, q** $\Rightarrow$ P : 製品の集合

✓ **原料 m, n** $\Rightarrow$ M : 原料の集合

数理最適化モデルの一般化②

■ 集合を用いて定数と変数を定義する

【問題】原料 m と n を用いて、製品 p と q を製造する。

次の表を参考に利得を最大にする製品 p と q の生産量をもとめよ。

必要量	原料 m	原料 n
製品 p	1	2
製品 q	3	1

原料	在庫
m	30
n	40

製品	利得
p	1
q	2

■ 定数

- ✓ 在庫量 (原料ごと) $\Rightarrow stock_m \ (m \in M)$
- ✓ 必要量 (製品と原料ごと) $\Rightarrow require_{p,m} \ (p \in P, m \in M)$
- ✓ 利得 (製品ごと) $\Rightarrow gain_p \ (p \in P)$

■ 変数

- ✓ 生産量 (製品ごと) $\Rightarrow x_p \ (p \in P)$

【Note】

P : 製品の集合

M : 原料の集合

数理最適化モデルの一般化③

- 集合と定数と変数を用いて制約式と目的関数を定義する

【問題】原料 m と n を用いて、製品 p と q を製造する。

次の表を参考に利得を最大にする製品 p と q の生産量をもとめよ。

必要量	原料 m	原料 n
製品 p	1	2
製品 q	3	1

原料	在庫
m	30
n	40

製品	利得
p	1
q	2

- 制約式

$x_p \geq 0$ ($p \in P$) : 製品 p の生産量は 0 以上

$\sum_{p \in P} require_{p,m} \cdot x_p \leq stock_m$ ($m \in M$) : 生産は在庫の範囲で行う

- 目的関数

$\sum_{p \in P} gain_p \cdot x_p$: 利得の最大化

数理最適化モデルの一般化

線形計画問題

- 集合
 - P : 製品の集合
 - M : 原料の集合
- 定数
 - $stock_m$ ($m \in M$) : 原料 m の在庫量
 - $require_{p,m}$ ($p \in P, m \in M$) : 製品 p を生産するのに必要な原料 m の量
 - $gain_p$ ($p \in P$) : 製品 p を生産した際の利得
- 変数
 - x_p ($p \in P$) : 製品 p の生産量
- 制約式
 - $x_p \geq 0$ ($p \in P$) : 製品 p の生産量は 0 以上
 - $\sum_{p \in P} require_{p,m} \cdot x_p \leq stock_m$ ($m \in M$) : 生産は在庫の範囲で行う
- 目的関数 (最大化)
 - $\sum_{p \in P} gain_p \cdot x_p$: 利得の最大化

数理モデルから
数値が排除されていることを確認

問題(=入力データ)変更への対応

【問題】原料 m と n を用いて、製品 p と q を製造する。
次の表を参考に利得を最大にする製品 p と q の生産量をもとめよ。

必要量	原料 m	原料 n
製品 p	1	2
製品 q	3	1

原料	在庫
m	30
n	40

製品	利得
p	1
q	2

問題(=入力データ)を変更して集合の要素を増やす

製品の集合： $P = \{p1, p2, p3, p4\}$

原料の集合： $M = \{m1, m2, m3\}$

入力データの一般化

■ 縦持ちのデータとしてCSVファイルを作成

require.csv

p,m,require

p1,m1,2

p1,m2,0

p1,m3,1

p2,m1,3

p2,m2,2

p2,m3,0

p3,m1,0

p3,m2,2

p3,m3,2

p4,m1,2

p4,m2,2

p4,m3,2

p:製品名

m:原料名

require:必要量

stock.csv

m,stock

m1,35

m2,22

m3,27

m:原料名

stock:在庫量

gain.csv

p,gain

p1,3

p2,4

p3,4

p4,5

p:製品名

gain:利益

※ 製品と原料の集合

製品の集合: $P = \{p1, p2, p3, p4\}$

原料の集合: $M = \{m1, m2, m3\}$

PuLPによる実装(ファイル入力)①

```
import pandas as pd
import pulp

# データの取得
stock_df = pd.read_csv('stock.csv')
gain_df = pd.read_csv('gain.csv')
require_df = pd.read_csv('require.csv')

# 集合の定義
P = gain_df['p'].tolist()
M = stock_df['m'].tolist()

# 定数の定義
m2stock = {row.m:row.stock
            for row in stock_df.itertuples()}
p2gain = {row.p:row.gain
          for row in gain_df.itertuples()}
pm2require = {(row.p,row.m):row.require
              for row in require_df.itertuples()}
```

PuLPによる実装(ファイル入力)②

```
# 最大化問題として数理モデルを定義
```

```
prob = pulp.LpProblem('LP', pulp.LpMaximize)
```

```
# 変数の定義
```

```
x = pulp.LpVariable.dicts('x', P, lowBound=0, cat='Continuous')
```

```
# 制約式の定義
```

```
for m in M:
```

```
    prob += pulp.lpSum([pm2require[p,m] * x[p] for p in P]) <= m2stock[m]
```

```
# 目的関数の定義
```

```
prob += pulp.lpSum([p2gain[p] * x[p] for p in P])
```

```
# 求解
```

```
status = prob.solve()
```

```
print('Status:', pulp.LpStatus[status])
```

```
for p in P:
```

```
    print(p, x[p].value())
```

```
print('obj=', prob.objective.value())
```

```
Status: Optimal
```

```
p1 12.142857
```

```
p2 3.5714286
```

```
p3 7.4285714
```

```
p4 0.0
```

```
obj= 80.42857099999999
```

解の考察

■ 解の出力結果

- ✓ 利益は**80.4** 万円
- ✓ 製品p1を**12.14** [kg]
- ✓ 製品p2を**3.57** [kg]
- ✓ 製品p3を**7.43** [kg]
- ✓ 製品p4を生産しない

```
Status: Optimal  
p1 12.142857  
p2 3.5714286  
p3 7.4285714  
p4 0.0  
obj= 80.42857099999999
```

- ## ■ 生産量が小数点以下を許さない場合どのようにすればよいか？ (単位が[kg]ではなく、[個]などの場合によくある) ⇒ 変数の定義域を**実数**から**整数**へ変更する

整数計画問題へ

- 線形計画問題から整数計画問題への変更は1行の修正が必要

変数の定義

```
x = pulp.LpVariable.dicts('x', P, lowBound=0, cat='Continuous')
```

から次へ変更

変数の定義

```
x = pulp.LpVariable.dicts('x', P, lowBound=0, cat='Integer')
```

- 解の出力は次のように変わる

- ✓ 利益は79万円
- ✓ 製品p1を13[kg]
- ✓ 製品p2を3[kg]
- ✓ 製品p3を7[kg]
- ✓ 製品p4を生産しない

Status: Optimal

p1 13.0

p2 3.0

p3 7.0

p4 0.0

obj= 79.0

整数計画問題の注意点

- 整数計画問題は、線形計画問題に比べて難しく、変数が少ないケースしか解けない場合がある
 - ✓ 例えば、線形計画問題であれば100万変数くらい解けそうなものだが、整数計画問題であれば1000変数でも解けない場合もある（ただし、ソルバーによる）
- 実践的には整数計画問題を線形計画問題として解いて（線形緩和）、解の小数点以下を丸める場合が多い
- 実務では現実的に解けない整数計画問題が多く、列生成法やラグランジュ緩和を利用した大規模解法が組まれることがある

まとめ

まとめ

- プログラミング言語Pythonを利用して数理最適化問題を解く方法を説明した。
- Pythonライブラリ PuLP を利用して次の問題を解いた
 - ✓ 連立一次方程式
 - ✓ 線形計画問題
 - ✓ 整数計画問題
- 数理最適化モデルを一般化することで、
コード量を減らす 効果や**汎用性を高める** ことができる。
(モデルとデータの分離)
- 数式やコードを言語化することが重要

演習

課題

- 課題をmanabaのレポートから取得する
 - ✓ `kadai_1.ipynb`
- 課題に取り組む
 - ✓ `kadai_1.ipynb` の課題を解く。
- 課題を提出する
 - ✓ manabaで課題を提出する。ただし、ファイル名は、
[学籍番号][氏名]_kadai_1.ipynb
とする。
 - ✓ 提出期限は2026.4.21(火)23:55とする。

次回の講義予告

第2回、第3回の講義について

- 岩永担当2回目 4.28(火)15:15-18:00
 - ✓ 教科書「Pythonではじめる数理最適化」3章
- 岩永担当3回目 5.12 (火)15:15-18:00
 - ✓ 教科書「Pythonではじめる数理最適化」4章～7章から1つ選択